

دوره آفلاین یادگیری ماشین های بدون نظارت

دوره آموزشی: یادگیری ماشین بدون نظارت
تاریخ گزارش: ۲۴ تیر تا ۳۰ تیر ۱۴۰۵

مدرس: جناب آقای مصطفی توسلی پور
نگارنده: سیدعرفان مسعودی

@erfanmasoudiba

erfanmasoudiba@gmail.com

linkedin.com/in/erfan-masoudi github.com/ErfanMasoudiBA

فهرست مطالب

۲	۱ تمرین اول - سوال اول: فشردن سازی تصویر با K-Means
۵	۲ تمرین اول - سؤال دوم: خوشه بندی با DBSCAN
۸	۳ تمرین اول - سؤال سوم: خوشه بندی سلسله مراتب
۱۱	۴ تمرین اول - سؤال چهارم: خوشه بندی ترکیبی گوسی (GMM)
۱۵	۵ تمرین دوم - سؤال اول: کاهش بعد با PCA و LDA
۱۸	۶ تمرین دوم - سؤال دوم: بیشینه درست نمایی (MLE) با توزیع لاپلاس
۲۱	۷ تمرین دوم - سؤال سوم: بررسی نفرین ابعاد
۲۳	۸ تمرین دوم - سؤال چهارم: انتخاب ویژگی افزایشی Selection Forward
۲۶	۹ تمرین سوم - سوال اول: پیاده سازی Autoencoder Variational روی مجموعه داده CelebA
۳۴	۱۰ تمرین سوم - سوال دوم: ترکیب VAE و Flows Normalizing
۳۸	۱۱ تمرین سوم - سوال سوم: یادگیری خودنظارتی (SSL)

۱ تمرین اول - سوال اول: فشرده‌سازی تصویر با الگوریتم K-Means

هدف و معرفی مسئله

هدف این تمرین، پیاده‌سازی الگوریتم K-Means Clustering از ابتدا و استفاده از آن برای فشرده‌سازی تصاویر بود. در این روش، هر پیکسل تصویر به عنوان یک نمونه سه‌بعدی شامل مؤلفه‌های رنگی RGB در نظر گرفته شده و با خوشه‌بندی این نمونه‌ها، تعداد رنگ‌های تصویر کاهش می‌یابد. در نهایت هر پیکسل با رنگ مرکز خوشه‌ی متناظر جایگزین می‌شود و تصویری با تعداد رنگ کمتر تولید خواهد شد.

پیاده‌سازی الگوریتم K-Means

الگوریتم K-Means به صورت کامل و بدون استفاده از توابع آماده پیاده‌سازی شد. مراحل اجرای الگوریتم شامل موارد زیر است:

- محاسبه فاصله اقلیدسی هر نمونه تا تمامی مراکز خوشه
 - اختصاص هر نمونه به نزدیک‌ترین مرکز خوشه
 - محاسبه مجدد مراکز خوشه با استفاده از میانگین اعضای هر خوشه
 - تکرار مراحل فوق تا رسیدن به همگرایی یا پایان تعداد تکرارها
- برای محاسبه فاصله بین نمونه‌ها و مراکز خوشه از فاصله اقلیدسی استفاده شد:

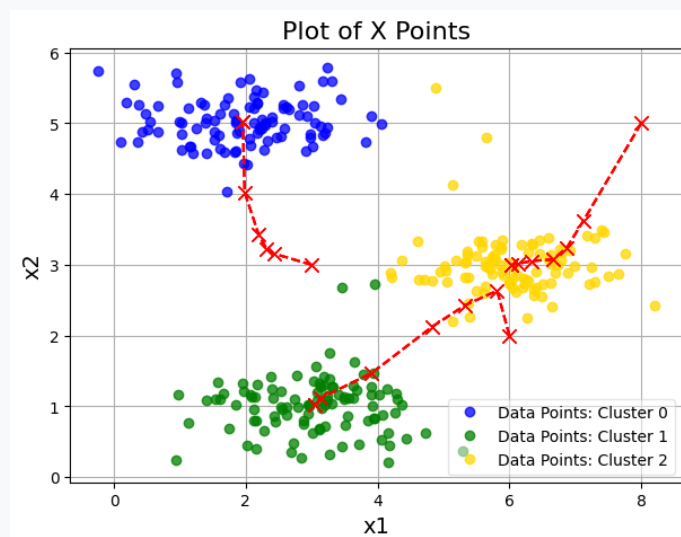
$$d(x, c) = \sum_{i=1}^n (x_i - c_i)^2$$

که در آن x نمونه داده و c مرکز خوشه است.

اجرای الگوریتم روی مجموعه داده نمونه

در مرحله نخست، عملکرد الگوریتم روی مجموعه داده دوبعدی ارائه‌شده در تمرین بررسی شد. سه مرکز اولیه به صورت دستی انتخاب شدند و الگوریتم طی چند تکرار اجرا گردید. همان‌طور که در شکل مشاهده می‌شود، مراکز خوشه در هر تکرار به

سمت میانگین داده‌های اختصاص یافته حرکت کرده و در نهایت به یک موقعیت پایدار همگرا شدند.



شکل ۱: همگرایی مراکز خوشه در اجرای الگوریتم K-Means

◀ فشرده‌سازی تصویر

برای بررسی کاربرد عملی خوشه‌بندی، یک تصویر رنگی با ابعاد 128×128 انتخاب شد. ابتدا مقادیر رنگی تصویر به بازه $[0, 1]$ نرمال شدند و سپس تصویر سه‌بعدی به ماتریسی با ابعاد

$$3 \times 16384$$

تبدیل گردید؛ به طوری که هر سطر نماینده یک پیکسل و هر ستون بیانگر یکی از مؤلفه‌های رنگی RGB بود.

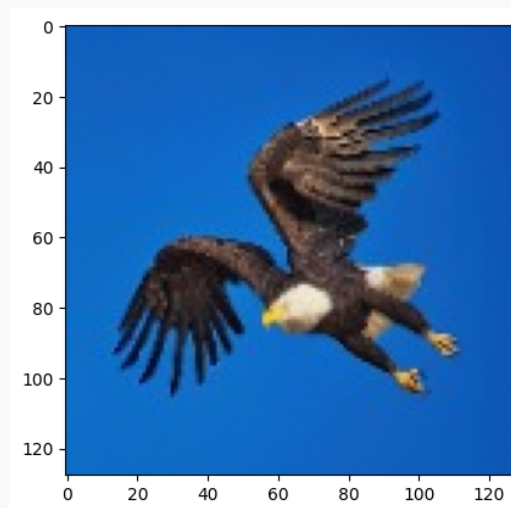
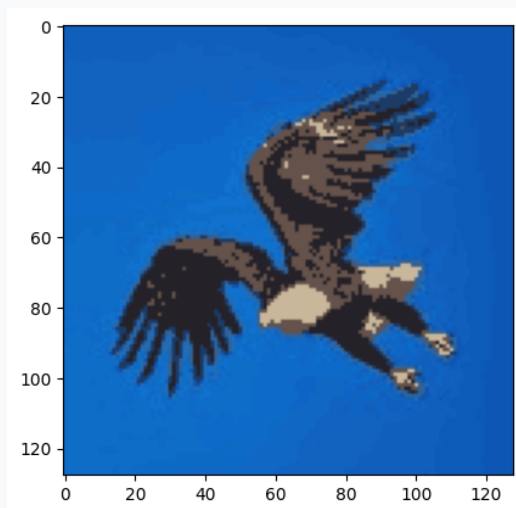
پس از آن، الگوریتم K-Means با تعداد خوشه

$$K = 16$$

اجرا شد. در پایان، هر پیکسل با رنگ مرکز خوشه متناظر جایگزین شد و تصویر فشرده‌شده تولید گردید.

نتایج و تحلیل

شکل زیر تصویر اصلی و تصویر فشرده شده را نشان می دهد. همان طور که مشاهده می شود، با وجود کاهش تعداد رنگ ها به تنها ۱۶ رنگ، ساختار اصلی تصویر همچنان حفظ شده است و تنها بخشی از جزئیات رنگی از بین رفته اند.



شکل ۲: مقایسه تصویر اصلی و تصویر حاصل از فشرده سازی با الگوریتم K-Means

کاهش تعداد رنگ ها باعث کاهش اطلاعات مورد نیاز برای ذخیره سازی تصویر می شود. هرچه مقدار K افزایش یابد، کیفیت تصویر نهایی بیشتر خواهد شد؛ اما میزان فشرده سازی کاهش پیدا می کند. در مقابل، انتخاب مقادیر کوچک تر برای K موجب کاهش بیشتر حجم داده ها خواهد شد اما افت کیفیت تصویر نیز محسوس تر خواهد بود.

جمع بندی

در این تمرین، الگوریتم K-Means از پایه پیاده سازی و سپس در مسئله فشرده سازی تصویر به کار گرفته شد. نتایج نشان دادند که خوشه بندی رنگ ها می تواند بدون از بین رفتن ساختار کلی تصویر، تعداد رنگ های مورد استفاده را به طور قابل توجهی کاهش دهد. این آزمایش یکی از کاربردهای مهم الگوریتم های یادگیری بدون نظارت در پردازش تصویر را به خوبی نشان می دهد.

۲ تمرین اول - سؤال دوم: خوشه‌بندی داده‌ها با الگوریتم DBSCAN

هدف و معرفی مسئله

هدف این بخش، پیاده‌سازی الگوریتم خوشه‌بندی مبتنی بر چگالی (DBSCAN) و بررسی عملکرد آن روی یک مجموعه داده دوبعدی بود. برخلاف الگوریتم K-Means که تعداد خوشه‌ها باید از قبل مشخص شود، الگوریتم DBSCAN با استفاده از چگالی نقاط، خوشه‌ها را به صورت خودکار شناسایی کرده و همچنین قادر به تشخیص نقاط نویز نیز می‌باشد.

مجموعه داده مورد استفاده شامل ۴۴۰۶ نمونه و دو ویژگی عددی بود که خروجی الگوریتم t-SNE را نمایش می‌دهد و برای انجام خوشه‌بندی مناسب است.

تعیین پارامترهای الگوریتم

الگوریتم DBSCAN به دو پارامتر اصلی نیاز دارد:

• ϵ : شعاع همسایگی هر نقطه

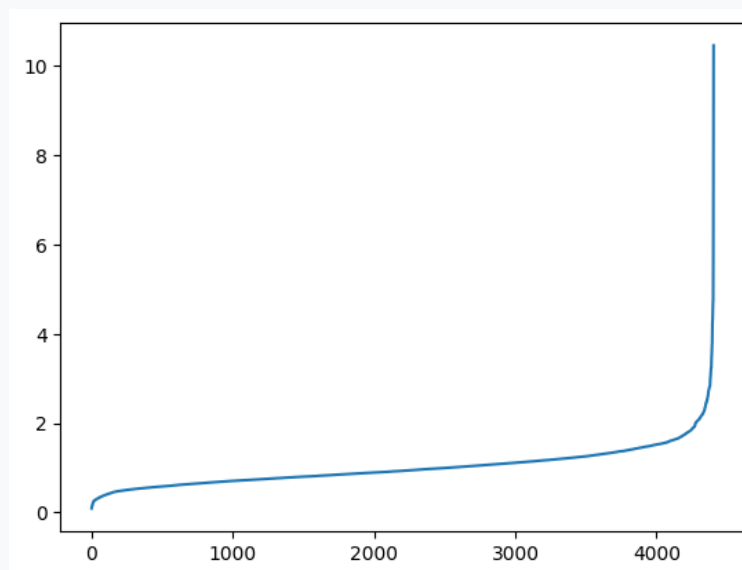
• MinPts : حداقل تعداد نقاط لازم برای تشکیل یک ناحیه چگال

مطابق پیشنهاد رایج برای داده‌های دوبعدی، مقدار

$$\text{MinPts} = 2 \times d = 4$$

در نظر گرفته شد که در آن d تعداد ابعاد داده است.

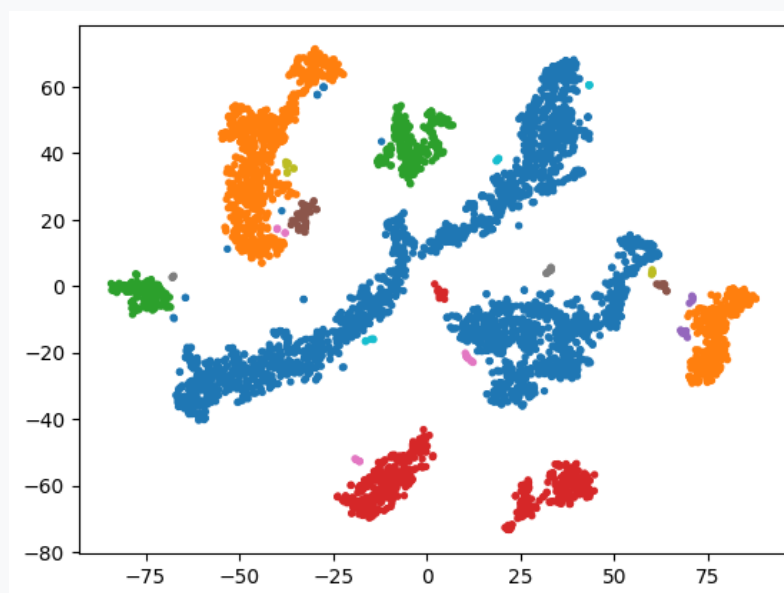
برای تعیین مقدار مناسب ϵ ابتدا فاصله چهارمین همسایه نزدیک هر نمونه با استفاده از الگوریتم KNN محاسبه شد. سپس با رسم نمودار فاصله‌ها و استفاده از الگوریتم KneeLocator، نقطه زانویی منحنی استخراج و مقدار متناظر آن به عنوان مقدار مناسب ϵ انتخاب گردید.



شکل ۳: نمودار فاصله چهارمین همسایه و تعیین مقدار مناسب ϵ

◀ اجرای الگوریتم DBSCAN

پس از تعیین پارامترهای الگوریتم، مدل DBSCAN با استفاده از کتابخانه Scikit-Learn روی مجموعه داده اجرا شد. در پایان، هر نمونه به یکی از خوشه‌ها اختصاص داده شد و نقاطی که در هیچ ناحیه چگالی قرار نگرفتند، به عنوان نویز (Noise) با برچسب ۱- مشخص شدند.



شکل ۴: نتیجه خوشه‌بندی داده‌ها توسط الگوریتم DBSCAN

نتایج و تحلیل

نتایج اجرای الگوریتم نشان داد که DBSCAN موفق به شناسایی

۲۳

خوشه مختلف در مجموعه داده شده است. همچنین تعداد

۳۴

نمونه به عنوان نقاط نویز شناسایی شدند که در هیچ ناحیه با چگالی کافی قرار نداشتند.

مزیت اصلی این الگوریتم نسبت به روش‌هایی مانند K-Means، عدم نیاز به تعیین تعداد خوشه‌ها و همچنین توانایی تشخیص نقاط پرت است. این ویژگی باعث می‌شود DBSCAN برای داده‌هایی با شکل‌های پیچیده و نامنظم عملکرد بسیار مناسبی داشته باشد.

جمع‌بندی

در این تمرین الگوریتم DBSCAN با تعیین خودکار مقدار ϵ به کمک نمودار فاصله همسایه‌ها و الگوریتم KneeLocator پیاده‌سازی شد. نتایج نشان داد که این روش قادر است بدون اطلاع قبلی از تعداد خوشه‌ها، ساختار طبیعی داده‌ها را استخراج کرده و نقاط نویز را نیز به خوبی شناسایی نماید. به همین دلیل DBSCAN یکی از مناسب‌ترین روش‌های خوشه‌بندی برای داده‌های با توزیع نامنظم و دارای نقاط پرت محسوب می‌شود.

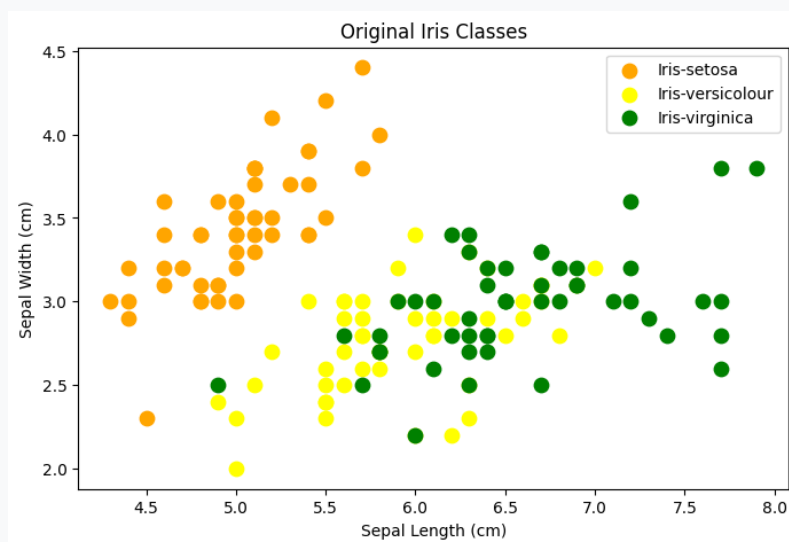
۳ تمرین اول - سؤال سوم: خوشه‌بندی داده‌های IRIS با الگوریتم Clustering Agglomerative

هدف و معرفی مسئله

هدف این بخش، پیاده‌سازی و ارزیابی الگوریتم خوشه‌بندی سلسله‌مراتب تجمعی (Agglomerative Hierarchical Clustering) روی مجموعه داده معروف IRIS بود. این مجموعه داده شامل ۱۵۰ نمونه از سه گونه مختلف گل زنبق (Setosa، Versicolor و Virginica) بر اساس ۴ ویژگی طول و عرض کاسبرگ و گلبرگ است. در این تمرین، فرآیند خوشه‌بندی به صورت کاملاً بدون نظارت (Unsupervised) و با حذف برچسب‌های واقعی کلاس‌ها انجام شد تا توانایی الگوریتم در کشف ساختار طبیعی داده‌ها سنجیده شود.

تحلیل داده‌ها و بصری‌سازی اولیه

ابتدا داده‌های مربوط به هر یک از سه گونه گل به صورت مجزا فیلتر شده و توزیع ویژگی‌های آن‌ها بررسی گردید. سپس جهت مقایسه اولیه، نمودار پراکندگی ۲ بعدی نمونه‌ها بر اساس طول و عرض کاسبرگ برای تمامی کلاس‌های واقعی ترسیم شد.



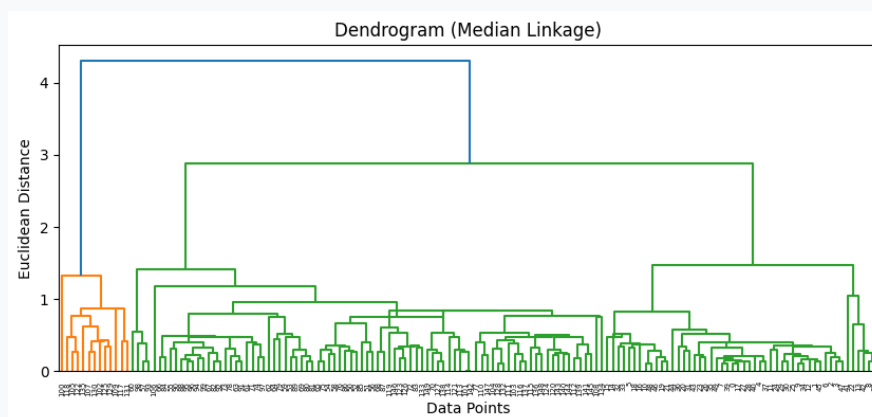
شکل ۵: پراکندگی واقعی گونه‌های مختلف گل IRIS در فضای ۲ بعدی

تحلیل دندروگرام و تعیین نوع پیوند (Linkage)

برای تعیین نحوه ادغام خوشه‌ها، دندروگرام مربوط به داده‌ها با چهار روش پیوند مختلف شامل Average، Complete، Single و Median رسم گردید. دندروگرام یک نمودار درختی است که نحوه ادغام تدریجی داده‌ها را در فواصل مختلف نشان می‌دهد. بر اساس ساختار دندروگرام و دانش اولیه از مسئله، تعداد خوشه‌ها برابر با

$$k = 3$$

تعیین شد و معیارهای پیوند میان خوشه‌ها ارزیابی گردیدند که روش Average Linkage بهترین پایداری و تفکیک‌پذیری را نشان داد.



شکل ۶: نمودار دندروگرام حاصل از خوشه‌بندی سلسله‌مراتب روی داده‌های IRIS

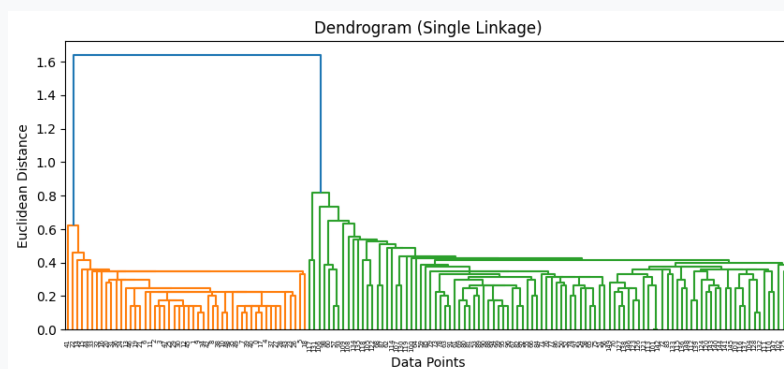
◀ اجرای الگوریتم و ارزیابی با ضریب سیلوئت

مدل AgglomerativeClustering با تعیین $n_clusters = 3$ و نوع پیوند average روی ویژگی‌های داده اجرا شد. جهت ارزیابی کیفیت خوشه‌بندی، معیار **ضریب سیلوئت (Silhouette Score)** محاسبه گردید.

مقدار ضریب سیلوئت به دست آمده برابر است با:

$$\text{Score Silhouette} \approx 0.55$$

این مقدار مثبت و نسبتاً بالا نشان‌دهنده ساختار مناسب خوشه‌ها و جدایش قابل قبول آن‌ها از یکدیگر است.



شکل ۷: نتایج حاصل از خوشه‌بندی داده‌های IRIS توسط الگوریتم Agglomerative Clustering

نتایج و تحلیل مقایسه‌ای

تحلیل و مقایسه خوشه‌های تشکیل‌شده توسط الگوریتم با برچسب‌های واقعی داده‌ها نتایج زیر را نشان می‌دهد:

- **گونه Setosa:** به دلیل فاصله زیاد ویژگی‌های آن از دو گونه دیگر، به صورت کاملاً مجزا و با دقت ۱۰۰٪ در یک خوشه مستقل قرار گرفت.

- **گونه‌های Versicolor و Virginica:** به دلیل همپوشانی طیف ویژگی‌های ظاهری در برخی نمونه‌ها، مرز میان این دو گونه دارای اندکی تداخل است که در خروجی الگوریتم خوشه‌بندی نیز این همپوشانی به خوبی مشهود می‌باشد.

جمع‌بندی

در این بخش الگوریتم خوشه‌بندی سلسله‌مراتب روی مجموعه داده IRIS پیاده‌سازی شد. با تحلیل نمودار دندروگرام و اعمال روش پیوند Average، الگوریتم موفق شد بدون دسترسی به برچسب‌های واقعی، سه خوشه اصلی داده‌ها را با ضریب سیلوئت ۰/۵۵ بازسازی کند. نتایج نشان داد که خوشه‌بندی سلسله‌مراتب گزینه‌ای بسیار مناسب برای درک ساختار درختی و ارتباطات میان داده‌ها است.

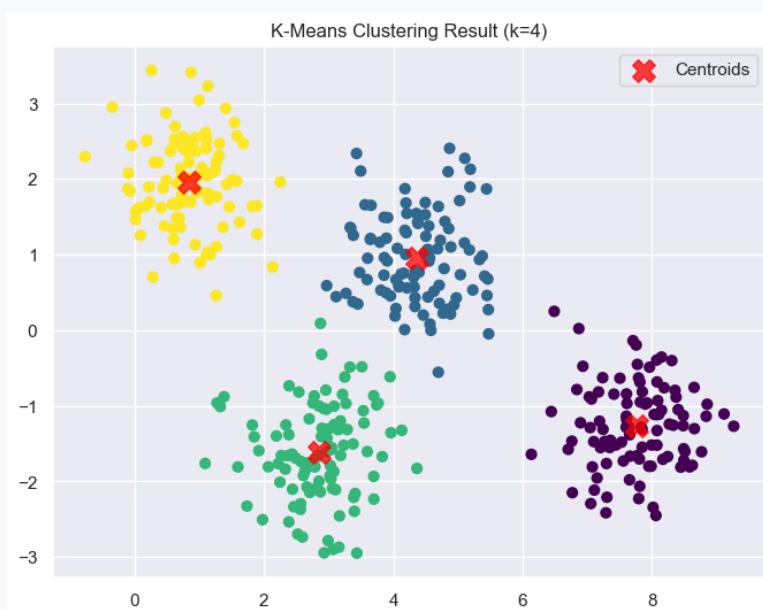
۴ تمرین اول - سؤال چهارم: خوشه‌بندی نرم داده‌ها با مدل ترکیبی گوسی (GMM)

هدف و معرفی مسئله

هدف این بخش، پیاده‌سازی و بررسی الگوریتم **Gaussian Mixture Model (GMM)** و مقایسه مفاهیم خوشه‌بندی سخت (Hard Clustering) و مانند K-Means با خوشه‌بندی نرم (Soft Clustering) می‌باشد. در الگوریتم‌های خوشه‌بندی نرم، هر نمونه برعکس الگوریتم‌های معمولی به صورت قطعی به یک خوشه متصل نمی‌شود، بلکه یک توزیع احتمالی از میزان تعلق نقطه به هر یک از توابع گوسی (خوشه‌ها) محاسبه می‌گردد. مجموعه داده مورد استفاده در این تمرین شامل ۴۰۰ نمونه دوبعدی تولیدشده توسط تابع `make_blobs` است.

ارزیابی اولیه و خوشه‌بندی با K-Means

با بررسی نمودار پراکندگی اولیه داده‌ها، وجود ۴ مرکز توزیع واضح در فضای داده مشاهده گردید. در مرحله نخست، الگوریتم K-Means با انتخاب $k = 4$ اجرا گردید. الگوریتم K-Means با فرض کروی بودن مرز خوشه‌ها و تخصیص قطعی مرزها عمل می‌کند. نتایج این خوشه‌بندی در شکل زیر آورده شده است.

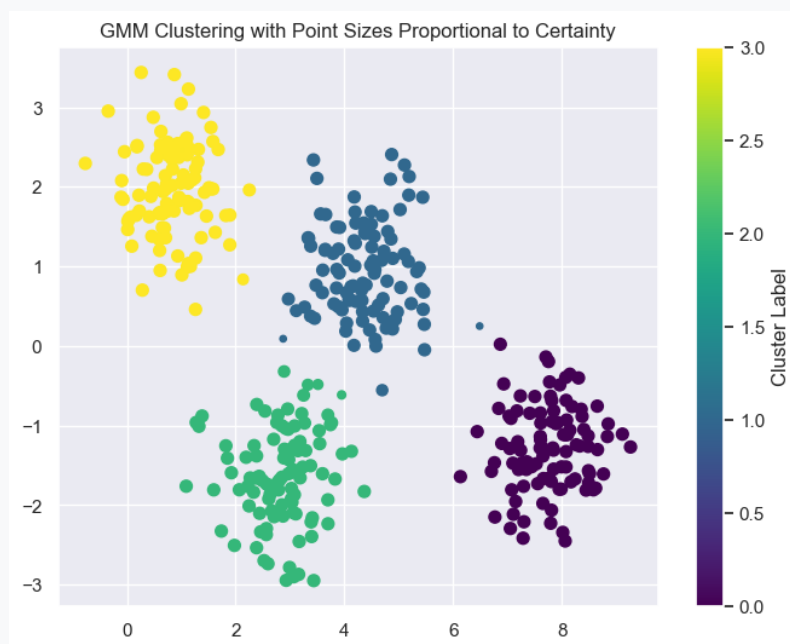


شکل ۸: نتیجه خوشه‌بندی داده‌ها توسط الگوریتم K-Means ($k = 4$)

آموزش مدل GMM و محاسبه ماتریس احتمالات

در ادامه، مدل GaussianMixture از ماژول sklearn.mixture با تعیین $n_components = 4$ روی داده‌ها برازش داده شد. سپس با استفاده از متد `predict_proba`، ماتریس احتمالات به ابعاد $(400, 4)$ محاسبه گردید.

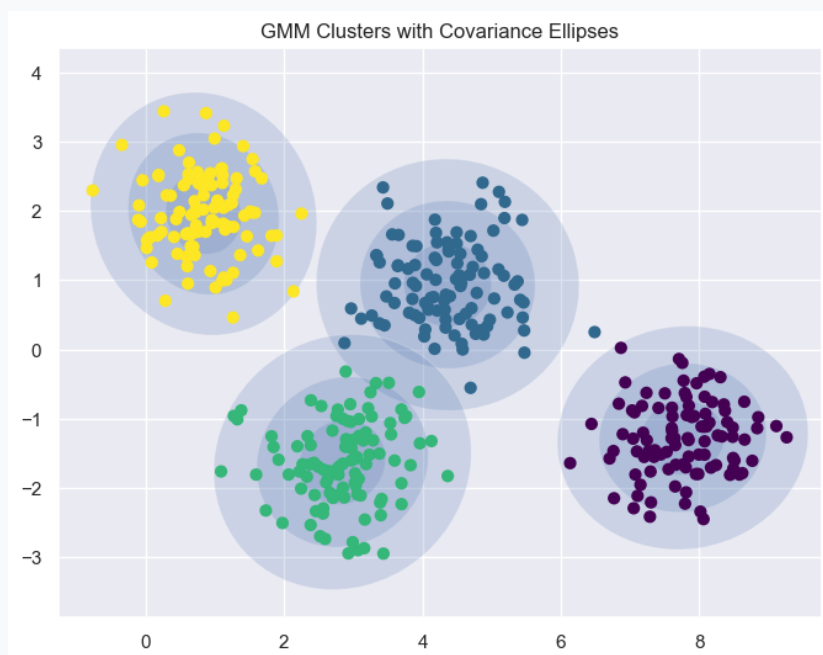
هر سطر از این ماتریس، احتمال حضور یک نمونه داده را در ۴ تابع توزیع گوسی نشان می‌دهد. جهت ملموس‌تر شدن مفهوم عدم قطعیت (Uncertainty)، اندازه نقاط در نمودار متناسب با حداکثر احتمال تعلق آن‌ها تنظیم گردید:



شکل ۹: نمایش میزان قطعیت خوشه‌بندی نمونه‌ها (اندازه بزرگتر نشان‌دهنده احتمال تعلق بالا به خوشه است)

رسم بیضی‌های کواریانس و تحلیل خروجی

با تکمیل توابع `draw_ellipse` و `plot_gmm`، بیضی‌های هم‌احتمال توزیع‌های گوسی برای هر خوشه ترسیم شدند. این بیضی‌ها ماتریس کواریانس و نحوه پراکندگی داده‌های هر خوشه را نمایش می‌دهند.



شکل ۱۰: نتیجه خوشه‌بندی GMM به همراه بیضی‌های توزیع کواریانس خوشه‌ها

نتایج و استنباط تحلیلی

تحلیل و استنباط حاصل از خروجی‌های مدل GMM:

- ****خوشه‌بندی نرم در برابر سخت: **** برعکس K-Means که نقاط مرزی را مجبورا به یک خوشه نسبت می‌دهد، GMM عدم قطعیت نقاط مرزی را بر حسب احتمال بازگو می‌کند.
- ****شکل خوشه‌ها: **** استفاده از ماتریس کواریانس عمومی به GMM این امکان را می‌دهد تا خوشه‌های بیضوی و کشیده را نیز بهتر از K-Means پوشش دهد.
- ****میزان انحراف: **** بیضی‌های مرکز خوشه‌ها فشرده‌تر و بیضی‌های بیرونی نشان‌دهنده انحراف معیارهای بالاتر (σ , 2σ , 3σ) در توزیع گوسی متناظر هستند.

جمع‌بندی

در این بخش، الگوریتم GMM با ۴ مؤلفه گوسی پیاده‌سازی و ارزیابی شد. نتایج نشان داد که این الگوریتم به واسطه ارائه خروجی احتمالی (`predict_proba`) و انعطاف‌پذیری

هندسی به کمک ماتریس کواریانس، ابزاری بسیار قدرتمندتر نسبت به K-Means برای تحلیل داده‌هایی با مرزهای مبهم یا توزیع‌های بیضوی است.

۵ تمرین دوم - سؤال اول: پیاده‌سازی دستی و کتابخانه‌ای روش‌های کاهش بعد PCA و LDA

هدف و معرفی مسئله

هدف این بخش، پیاده‌سازی الگوریتم‌های ****تحلیل مؤلفه‌های اصلی (PCA)**** به عنوان یک روش کاهش بعد بدون نظارت (Unsupervised) و ****تحلیل ممیز خطی (LDA)**** به عنوان یک روش با نظارت (Supervised) بر روی مجموعه داده دیابت (diabetes.csv) می‌باشد.

در این سؤال، نحوه عملکرد الگوریتم‌ها با پیاده‌سازی ریاضی دستی توسط کتابخانه NumPy انجام شده و خروجی‌های بصری آن به صورت دقیق با خروجی‌های کتابخانه Scikit-Learn مقایسه و تحلیل گردید.

پیش‌پردازش داده‌ها و استانداردسازی

مجموعه داده دیابت شامل ۷۶۸ نمونه و ۸ ویژگی پزشکی (مانند قند خون، انسولین، فشار خون و سن) به همراه متغیر هدف Outcome است. از آنجا که روش‌های کاهش بعد مبتنی بر ماتریس کواریانس و فاصله‌ها هستند، ویژگی‌ها با استفاده از StandardScaler به میانگین صفر و واریانس یک استانداردسازی شدند:

$$z = \frac{x - \mu}{\sigma}$$

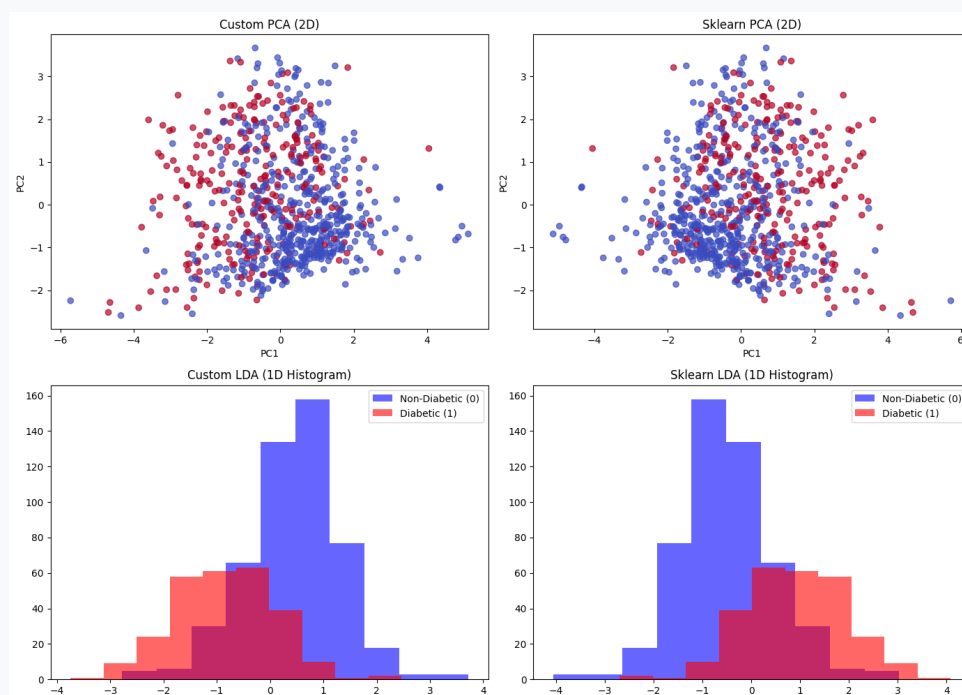
پیاده‌سازی دستی الگوریتم PCA و LDA

مراحل پیاده‌سازی ریاضی دو الگوریتم با NumPy به شرح زیر انجام شد:

- ****الگوریتم PCA****: با مرکزیک کردن داده‌ها، محاسبه ماتریس کواریانس و استخراج بردارهای ویژه (Eigenvectors)، ۲ مؤلفه اصلی برتر انتخاب شدند.
- ****الگوریتم LDA****: با محاسبه ماتریس‌های پراکندگی درون‌کلاسی (S_W) و بین‌کلاسی (S_B) و حل مسئله مقدار ویژه تعمیم‌یافته برای $S_W^{-1}S_B$ پیاده‌سازی شد. با توجه به $C = ۲$ کلاس، خروجی LDA به $C - ۱ = ۱$ بعد نگاشت گردید.

تحلیل بصری خروجی ها و مقایسه پیاده سازی ها

با بررسی دقیق چهار نمودار تولیدشده در خروجی کد، نتایج تحلیلی زیر استخراج گردید:



شکل ۱۱: مقایسه دقیق خروجی های پیاده سازی دستی (Custom) و کتابخانه ای (Sklearn) برای الگوریتم های PCA و LDA

- ****تحلیل معکوس شدن علامت در محورها (Sign Inversion):**** همان طور که در نمودار PCA و هیستوگرام LDA مشهود است، شکل کلی پراکندگی داده ها در پیاده سازی دستی و اسکالرن کاملاً یکسان است، با این تفاوت که جهت محور افقی (PC1) معکوس (180° چرخش) شده است. این موضوع از نظر ریاضی کاملاً طبیعی است؛ زیرا بردارهای ویژه (v) تنها تا ضرب یک ضرب کننده منفی ($\pm v$) یکتا هستند و هر دو جهت از نظر حفظ واریانس معتبرند.

- ****تحلیل عدم تفکیک پذیری در PCA:**** در نمودارهای دو بعدی PCA (تصاویر بالا)، داده های افراد دیابتی (قرمز) و غیردیابتی (آبی) به شدت در یکدیگر تداخل دارند. علت این امر عدم استفاده PCA از برچسب داده ها است؛ این الگوریتم صرفاً جهتی را انتخاب می کند که بیشترین پراکندگی (واریانس) را داشته باشد، نه جهتی که بهترین تفکیک کلاس را ایجاد کند.

• **تحلیل جدایش در LDA: در هیستوگرام های یک بعدی LDA (تصاویر پایین)، توزیع دو کلاس به میزان قابل توجهی از هم تفکیک شده اند. مرکز توزیع بیماران غیردیابتی و دیابتی در دو سمت مخالف محور قرار گرفته اند که نشان دهنده موفقیت LDA در بیشینه کردن فاصله میانگین کلاس ها نسبت به واریانس درون کلاسی است.

جمع بندی

در این بخش، صحت پیاده سازی ریاضی الگوریتم های PCA و LDA اثبات گردید. تحلیل بصری خروجی ها نشان داد که جهت گیری بردارهای ویژه می تواند تفاوت علامت جزئی داشته باشد اما رفتار کلی داده ها یکسان است. همچنین مشخص گردید برای مسائل دسته بندی و جداسازی کلاس ها، روش با نظارت LDA کارایی به مراتب بالاتری نسبت به روش بدون نظارت PCA ارائه می دهد.

۶ تمرین دوم - سؤال دوم: تخمین بیشینه درست‌نمایی (MLE) و رگرسیون لاپلاس

هدف و معرفی مسئله

هدف این بخش، پیاده‌سازی روش ****تخمین بیشینه درست‌نمایی (Maximum Likelihood Estimation - MLE)**** بر روی داده‌های زمان فوران آتشفشان (Q2.csv) با فرض تابع توزیع لاپلاس (Laplace Distribution) و مقایسه عملکرد آن با رگرسیون خطی معمولی (L_2 / OLS) می‌باشد.

در بسیاری از مسائل دنیای واقعی، داده‌ها ممکن است حاوی نقاط پرت (Outliers) باشند یا توزیع خطا غیرگوسی باشد. تابع توزیع لاپلاس به دلیل استفاده از قدر مطلق خطاها در لوگ-درست‌نمایی، مقاومت به مراتب بالاتری در برابر داده‌های پرت دارد.

تعریف ریاضی لگاریتم منفی درست‌نمایی (NLL)

فرمول منفی لگاریتم درست‌نمایی (Negative Log-Likelihood) برای n نمونه مستقل با توزیع لاپلاس به صورت زیر تعریف می‌شود:

$$\ell(y_1, y_2, \dots, y_n; \mu, \lambda) = - \sum_{i=1}^n \left(-\log(\lambda) - \frac{|y_i - \mu|}{\lambda} \right)$$

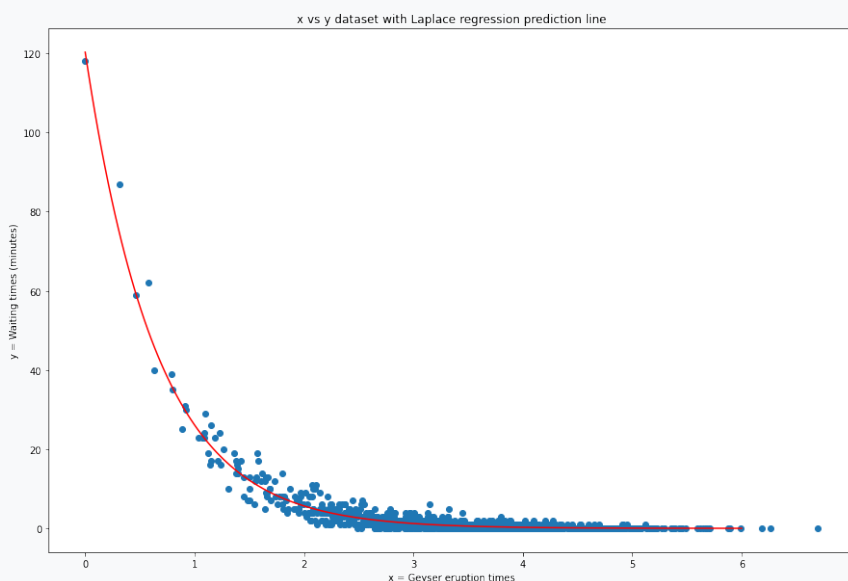
در مدل رگرسیون غیرخطی این تمرین، میانگین توزیع به صورت نمایی وابسته به ویژگی‌ها در نظر گرفته شد:

$$\mu = \exp(\mathbf{X}\mathbf{b})$$

جهت بهینه‌سازی و یافتن ضرایب $\mathbf{b}$ ، از الگوریتم بهینه‌سازی Powell درون تابع `scipy.optimize.minimize` استفاده گردید.

برآزش مدل لاپلاس بر روی داده‌های واقعی

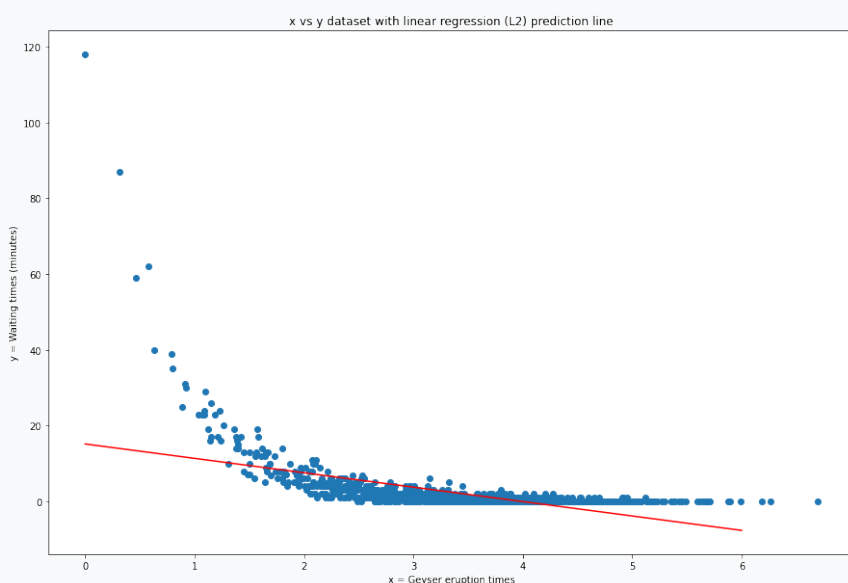
پس از تعریف توابع `'laplaceRegNegLogLikelihood'` و `'modelFit'`، ضرایب بهینه $\mathbf{b}$ برای داده‌های مجموعه Q2.csv استخراج شده و منحنی پیش‌بینی برای داده‌های جدید $x \in [0, 6]$ ترسیم گردید.



شکل ۱۲: برازش رگرسیون لاپلاس ($\mu = \exp(\mathbf{Xb})$) بر روی داده‌های زمان فوران و انتظار

➤ برازش رگرسیون خطی معمولی (OLS) / L_2 و مقایسه

در ادامه، مدل رگرسیون خطی معمولی با فرض توزیع خطا به صورت گوسی (کمترین مربعات) بر روی همان داده‌ها برازش داده شد.



شکل ۱۳: برازش رگرسیون خطی معمولی (OLS / L_2) بر روی مجموعه داده

◀ تحلیل و پاسخ به خواسته سوال (چرا رگرسیون خطی دچار مشکل است؟)

با مقایسه دو نمودار فوق، نتایج تحلیلی زیر استخراج گردید:

• ****ضعف رگرسیون خطی L_2 در برابر داده‌های پرت:** ****** در الگوریتم L_2 ، خطای هر نقطه به توان ۲ می‌رسد ($|y_i - \hat{y}_i|^2$). نقاط پرتی که مقدار y آن‌ها بسیار بزرگ است، جریمه فوق‌العاده شدیدی به تابع هزینه تحمیل می‌کنند. این امر باعث می‌شود که خط رگرسیون به شدت به سمت نقاط پرت منحرف شده و توانایی خود را در پیش‌بینی دقیق تنه اصلی داده‌ها از دست بدهد (Underfitting روی داده‌های اصلی).

• ****مزیت رگرسیون لاپلاس (L_1):** ****** توزیع لاپلاس از قدر مطلق خطاها ($|y_i - \hat{y}_i|$) استفاده می‌کند. این ویژگی باعث می‌شود که تاثیر نقاط پرت به صورت خطی رشد کند نه نمایی؛ در نتیجه مدل لاپلاس نسبت به نقاط پرت استوار (Robust) بوده و رفتار واقعی روندهای نمایی در داده‌ها را بسیار دقیق‌تر مدل‌سازی می‌کند.

◀ جمع‌بندی

در این تمرین، با پیاده‌سازی دستی تابع درست‌نمایی لاپلاس و بهینه‌سازی آن، نشان داده شد که انتخاب تابع توزیع مناسب متناسب با ماهیت داده‌ها اهمیت بالایی دارد. رگرسیون لاپلاس به دلیل استواری در برابر داده‌های پرت (Outlier Robustness) و فرم نمایی $\mu = \exp(Xb)$ ، عملکرد بسیار برتری نسبت به رگرسیون خطی معمولی ارائه داد.

۷ تمرین دوم - سؤال سوم: بررسی پدیده نفرین ابعاد (Curse of Dimensionality)

هدف و معرفی مسئله

هدف این بخش، بررسی تجربی یکی از چالش‌های بنیادین در یادگیری ماشین و پردازش داده تحت عنوان **نفرین ابعاد (Curse of Dimensionality)** است. با افزایش ابعاد داده‌ها، فضای حالت به قدری بزرگ و خلوت (Sparse) می‌شود که محاسبات مبتنی بر فاصله (نظیر k-NN یا خوشه‌بندی) دچار اختلال شدید شده و مفهوم «همسایگی نزدیک» معنای فیزیکی و آماری خود را از دست می‌دهد.

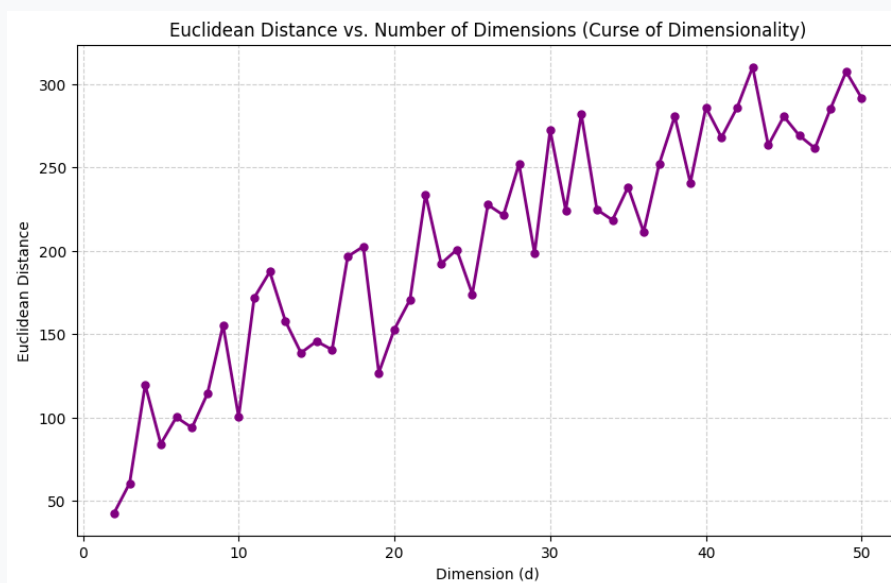
نحوه پیاده‌سازی و فرمول محاسباتی

برای بررسی این پدیده، در هر بعد d از ۲ تا ۵۰ ($d \in [2, 50]$) دو بردار تصادفی x و y با درایه‌های تصادفی صحیح در بازه $[0, 100]$ با استفاده از تابع `np.random.randint` ایجاد گردید. سپس فاصله اقلیدسی بین آن‌ها به صورت زیر محاسبه شد:

$$D(x, y) = \|x - y\|_2 = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$$

نمودار و تحلیل خروجی

نمودار زیر تغییرات فاصله اقلیدسی دو بردار تصادفی را بر حسب افزایش ابعاد از $d = 2$ تا $d = 50$ نشان می‌دهد:



شکل ۱۴: تغییرات فاصله اقلیدسی بر حسب تعداد ابعاد (d)

- ****** روند صعودی فاصله اقلیدسی: ****** همان طور که در نمودار مشخص است، با افزایش ابعاد d ، میانگین فاصله اقلیدسی بین دو نقطه تصادفی به شدت رشد کرده و از حدود ۴۰ در فضای ۲ بعدی به بیش از ۲۹۰ در فضای ۵۰ بعدی می‌رسد.
- ****** علت ریاضی: ****** هر بعد جدید، یک جمله مثبت $(x_i - y_i)^2$ به زیر رادیکال اضافه می‌کند که باعث افزایش مجموع مربع خطاهایت تحت جذر می‌شود.

استنباط و نتیجه‌گیری

تحلیل این خروجی، مفاهیم زیر را در یادگیری ماشین روشن می‌سازد:

۱. ****** دور شدن نقاط از یکدیگر در ابعاد بالا: ****** در فضاهای با ابعاد بسیار بالا، تمامی نقاط به طرز عجیبی از هم دور شده و فاصله‌های نسبی بین تمام جفت نقاط تقریباً یکسان می‌گردد.
۲. ****** ضرورت الگوریتم‌های کاهش بعد: ****** همین پدیده نشان می‌دهد چرا در داده‌های واقعی با ابعاد بالا، استفاده از روش‌هایی مانند PCA و LDA قبل از اعمال الگوریتم‌های یادگیری حیاتی و ضروری است.

۸. تمرین دوم - سؤال چهارم: پیاده‌سازی انتخاب ویژگی افزایشی (Forward Selection) بر روی سرطان سینه

هدف و معرفی مسئله

هدف این بخش، بررسی مسئله ****انتخاب ویژگی (Feature Selection)**** به روش ****افزایشی (Sequential Forward Selection - SFS)**** بر روی مجموعه داده سرطان سینه (cancer.csv) است.

هدف اصلی انتخاب ویژگی، کاهش ابعاد مسئله، حذف ویژگی‌های تکراری و نویز، جلوگیری از بیش‌برازش (Overfitting) و افزایش تفسیرپذیری مدل با حفظ حداکثر دقت دسته‌بندی می‌باشد.

بررسی اولیه داده‌ها و پاکسازی داده‌های پرت

مجموعه داده اولیه شامل ۵۶۹ نمونه، یک ستون شناسه (id)، ستون هدف diagnosis (با برچسب‌های M بدخیم و B خوش‌خیم) و ۳۰ ویژگی پزشکی عددی است. به منظور پالایش داده‌ها، با استفاده از روش محدوده بین‌چارکی (IQR)، نمونه‌های دارای مقادیر پرت شناسایی و حذف گردیدند. در نهایت ۳۹۸ نمونه پاکسازی‌شده برای مرحله آموزش و ارزیابی باقی ماندند.

پیاده‌سازی طبقه‌بند و انتخاب ویژگی افزایشی (SFS)

از مدل ****رگرسیون لجستیک (Logistic Regression)**** به عنوان طبقه‌بند پایه استفاده شد. روند اجرا به شرح زیر است:

۱. ****مدل کامل (با ۳۰ ویژگی):**** ابتدا مدل با تمامی ۳۰ ویژگی آموزش داده شد که به دقت آموزش ۹۵/۶٪ و دقت آزمون ۹۵/۰٪ دست یافت.

۲. ****اعمال SFS:**** با استفاده از ماژول SequentialFeatureSelector از کتابخانه Scikit-Learn، تعداد ۵ ویژگی برتر به صورت مرحله به مرحله انتخاب شدند.

ویژگی‌های منتخب عبارتند از:

• texture_mean (میانگین بافت)

• smoothness_mean (میانگین همواری)

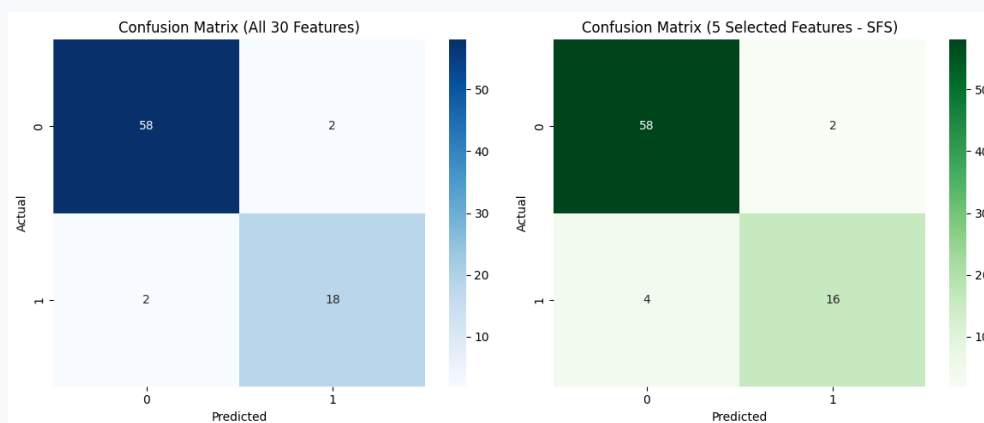
• perimeter_se (خطای معیار محیط)

• texture_worst (بدترین بافت)

• area_worst (بدترین مساحت)

◀ ارزیابی و مقایسه با ماتریس آشفستگی

ماتریس‌های آشفستگی (Confusion Matrix) برای دو حالت کامل (۳۰ ویژگی) و منتخب (۵ ویژگی SFS) در تصویر زیر رسم شده‌اند:



شکل ۱۵: مقایسه ماتریس آشفستگی مدل با کل ۳۰ ویژگی (سمت چپ) و ۵ ویژگی منتخب SFS (سمت راست)

• ****تحلیل ماتریس کامل (۳۰ ویژگی):**** از ۸۰ داده آزمون، ۵۸ نمونه منفی واقعی (True Negative) و ۱۸ نمونه مثبت واقعی (True Positive) به درستی پیش‌بینی شده‌اند و تنها ۴ خطای مجموع وجود دارد (دقت ۹۵٪).

• ****تحلیل ماتریس SFS (۵ ویژگی):**** با تنها ۵ ویژگی، تعداد ۵۸ نمونه منفی واقعی و ۱۶ نمونه مثبت واقعی به درستی تشخیص داده شده‌اند (دقت ۹۲٫۵٪).

• ****توجیه علت انتخاب این ویژگی‌ها:**** الگوریتم SFS ویژگی‌هایی را انتخاب کرده است که اطلاعات غیرتکراری ارائه می‌دهند؛ ترکیب شاخص‌های بافت

(texture) و اندازه خطیر غده (area_worst و perimeter_se) بیشترین قدرت تفکیک بافت های بدخیم و خوش خیم را فراهم ساخته است.

جمع بندی

نتایج نشان داد که با کاهش ۸۳٪ ابعاد ویژگی ها (از ۳۰ به ۵ ویژگی)، دقت مدل تنها ۲/۵٪ افت داشته است که نشان دهنده کارایی بالای الگوریتم Forward Selection در بهینه سازی مدل های پزشکی می باشد.

۹ ☺ تمرین سوم - سوال اول: پیاده‌سازی Au- Variational toencoder روی مجموعه داده CelebA

◀ هدف و معرفی مسئله

هدف این تمرین، پیاده‌سازی گام به گام یک Variational Autoencoder (VAE) بر روی مجموعه داده چهره‌های CelebA بود. برای درک بهتر تفاوت میان یک اتوانکودر معمولی و نسخه واریشنال آن، ابتدا یک Simple Autoencoder به طور کامل پیاده‌سازی و آموزش داده شد و سپس همان معماری با افزودن مکانیزم Reparameterization و جمله KL Divergence به یک VAE تبدیل گردید. در پایان، عملکرد دو مدل از نظر کیفیت بازسازی تصاویر و همچنین توانایی تولید چهره‌های جدید از فضای نهفته مقایسه شد.

◀ آماده‌سازی داده و پیکربندی اولیه

مجموعه داده CelebA با استفاده از Kaggle API دانلود و در محیط Colab استخراج شد. با توجه به حجم بالای داده، به جای بارگذاری کامل تصاویر در حافظه، از کلاس ImageDataGenerator و متد flow_from_directory برای خواندن تدریجی تصاویر از روی دیسک استفاده شد. این رویکرد باعث می‌شود مصرف حافظه در طول آموزش کنترل شده باقی بماند، هرچند به قیمت کاهش سرعت خواندن داده از دیسک در مقایسه با نگه داشتن کل داده در RAM تمام می‌شود. پارامترهای اصلی مدل مطابق خواسته سوال به شرح زیر تنظیم شدند:

• ابعاد ورودی تصاویر: $INPUT_DIM = (128, 128, 3)$

• اندازه بچ: $BATCH_SIZE = 512$

• ابعاد فضای نهفته (latent space): $Z_DIM = 200$

مقادیر پیکسلی تصاویر نیز با ضریب $1/255$ نرمال‌سازی شدند تا در بازه $[0, 1]$ قرار گیرند؛ این کار هم‌راستا با تابع فعال‌سازی Sigmoid در لایه خروجی دیکودر انتخاب شده است.

◀ تکمیل معماری انکودر

انکودر متشکل از چهار لایه Conv2D پشت سرهم است که هر کدام با $\text{stride}=2$ ابعاد مکانی تصویر را نصف می کنند؛ در نتیجه تصویر ورودی $3 \times 128 \times 128$ در نهایت به یک نگاشت ویژگی با ابعاد کوچک تر و عمق ۶۴ کانال تبدیل می شود. پس از هر لایه کانولوشنی از تابع فعال سازی LeakyReLU استفاده شده تا از مشکل Dying ReLU جلوگیری شود. در انتها، خروجی با Flatten به بردار یک بعدی تبدیل و توسط یک لایه Dense به بردار نهفته ای با طول $Z_DIM=200$ نگاشت می شود.

خلاصه معماری (خروجی `encoder.summary()`) نشان می دهد که تعداد پارامترهای قابل آموزش عمدتاً در لایه Dense انتهایی متمرکز است؛ زیرا این لایه، خروجی مسطح شده با ابعاد نسبتاً بزرگ را به بردار ۲۰۰ بعدی می نگارد و همین موضوع سهم بزرگی از حجم کل پارامترهای مدل را به خود اختصاص می دهد.

Model: "model"

Layer (type)	Output Shape	Param #
encoder_input (InputLayer)	[(None, 128, 128, 3)]	0
encoder_conv_0 (Conv2D)	(None, 64, 64, 32)	896
leaky_re_lu (LeakyReLU)	(None, 64, 64, 32)	0
encoder_conv_1 (Conv2D)	(None, 32, 32, 64)	18496
leaky_re_lu_1 (LeakyReLU)	(None, 32, 32, 64)	0
encoder_conv_2 (Conv2D)	(None, 16, 16, 64)	36928
leaky_re_lu_2 (LeakyReLU)	(None, 16, 16, 64)	0
encoder_conv_3 (Conv2D)	(None, 8, 8, 64)	36928
leaky_re_lu_3 (LeakyReLU)	(None, 8, 8, 64)	0
flatten (Flatten)	(None, 4096)	0
encoder_output (Dense)	(None, 200)	819400
...		
Total params: 912648 (3.48 MB)		
Trainable params: 912648 (3.48 MB)		
Non-trainable params: 0 (0.00 Byte)		

شکل ۱۶: خلاصه معماری انکودر اتوانکودر ساده

◀ تکمیل معماری دیکودر

دیکودر به گونه ای طراحی شده که تصویر آینه وار انکودر باشد. ابتدا بردار نهفته ۲۰۰ بعدی توسط یک لایه Dense به همان ابعاد فضایی ای که پیش از Flatten در انکودر وجود داشت (shape_before_flattening) بازگردانده و با Reshape به تانسور سه بعدی تبدیل می شود. سپس چهار لایه Conv2DTranspose با stride=2 به کار رفته تا ابعاد مکانی تصویر به تدریج دو برابر شده و در نهایت به ابعاد اصلی $۱۲۸ \times ۱۲۸ \times ۳$ بازگردد. در تمام لایه های میانی از LeakyReLU استفاده شده، اما لایه آخر با تابع فعال سازی Sigmoid همراه شده تا خروجی مدل در بازه $[۰, ۱]$ محدود بماند و با نرمال سازی انجام شده روی تصاویر ورودی سازگار باشد.

Model: "model_2"

Layer (type)	Output Shape	Param #
encoder_input (InputLayer)	[(None, 128, 128, 3)]	0
encoder_conv_0 (Conv2D)	(None, 64, 64, 32)	896
leaky_re_lu (LeakyReLU)	(None, 64, 64, 32)	0
encoder_conv_1 (Conv2D)	(None, 32, 32, 64)	18496
leaky_re_lu_1 (LeakyReLU)	(None, 32, 32, 64)	0
encoder_conv_2 (Conv2D)	(None, 16, 16, 64)	36928
leaky_re_lu_2 (LeakyReLU)	(None, 16, 16, 64)	0
encoder_conv_3 (Conv2D)	(None, 8, 8, 64)	36928
leaky_re_lu_3 (LeakyReLU)	(None, 8, 8, 64)	0
flatten (Flatten)	(None, 4096)	0
encoder_output (Dense)	(None, 200)	819400
...		
Total params: 1829131 (6.98 MB)		
Trainable params: 1829131 (6.98 MB)		
Non-trainable params: 0 (0.00 Byte)		

شکل ۱۷: خلاصه معماری دیکودر

◀ اتصال انکودر و دیکودر و آموزش اتوانکودر ساده

با اتصال خروجی انکودر به ورودی دیکودر، مدل کامل Simple Autoencoder ساخته شد. ورودی مدل ترکیبی همان ورودی انکودر و خروجی آن، خروجی دیکودر است؛ یعنی مدل یاد می‌گیرد تصویر ورودی را از مسیر یک گلوگاه ۲۰۰ بعدی عبور داده و دوباره بازسازی کند.

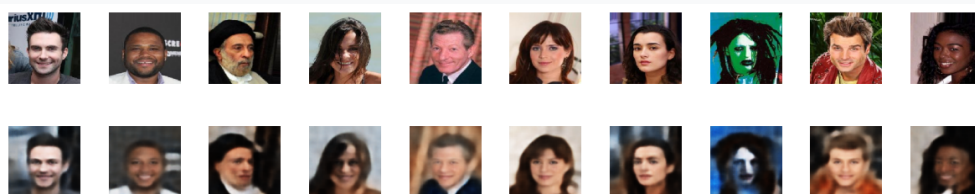
برای آموزش، از تابع خطای Mean Squared Error پیکسل‌به‌پیکسل به شکل زیر استفاده شد:

$$\mathcal{L}_r = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2$$

که در آن x تصویر واقعی و $\hat{x}$ تصویر بازسازی شده توسط مدل است. بهینه‌ساز Adam با نرخ یادگیری $\text{LEARNING_RATE} = 0.0005$ به کار رفت و مدل به مدت $N_EPOCHS = 10$ با استفاده از callback مربوط به ModelCheckpoint آموزش داده شد تا وزن‌های مدل پس از هر دوره ذخیره شوند.

◀ نتایج بازسازی تصاویر با اتوانکودر ساده

پس از آموزش، عملکرد مدل با مقایسه تصاویر اصلی و تصاویر بازسازی شده بررسی شد. همان‌طور که در شکل زیر مشاهده می‌شود، مدل توانسته ساختار کلی چهره‌ها از جمله موقعیت چشم‌ها، بینی، خطوط صورت و رنگ پوست را به خوبی بازتولید کند.



شکل ۱۸: ردیف بالا: تصاویر اصلی، ردیف پایین: تصاویر بازسازی شده توسط اتوانکودر ساده

با این حال، تصاویر بازسازی شده کمی محو (blurry) و فاقد جزئیات ریز مانند بافت مو یا لبه‌های تیز صورت هستند. این پدیده مستقیماً ناشی از انتخاب تابع خطای MSE است؛ از آنجا که این تابع خطا، میانگین تفاوت پیکسلی را کمینه می‌کند، در

حالت عدم قطعیت بین چند حالت ممکن برای یک پیکسل، مدل خروجی را به سمت میانگین آن حالت‌ها می‌برد که نتیجه‌اش محو شدن جزئیات پرفرکانس تصویر است. این محدودیت ذاتی اتوانکودرهای مبتنی بر MSE در برابر رویکردهایی مانند GAN که از یک شبکه تمایزدهنده برای واقعی‌تر کردن خروجی استفاده می‌کنند، به‌خوبی نمایان می‌شود.

◀ تبدیل به Autoencoder Variational

در گام بعد، انکودر بازطراحی شد تا به‌جای تولید مستقیم یک بردار نهفته قطعی، دو بردار `mean_mu` و `log_var` را تولید کند که به‌ترتیب میانگین و لگاریتم واریانس یک توزیع نرمال چندبعدی را در فضای نهفته توصیف می‌کنند. سپس با استفاده از ترفند Reparameterization Trick نمونه‌برداری از این توزیع به شکل قابل‌مشتق‌گیری انجام شد:

$$z = \mu + e^{\frac{1}{2}\log(\sigma^2)} \odot \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, I)$$

استفاده از این ترفند ضروری است، زیرا نمونه‌برداری تصادفی مستقیم غیرقابل‌مشتق‌گیری است و مانع از انتشار گرادیان در فرآیند Backpropagation می‌شود. با انتقال تصادفی بودن به متغیر کمکی ϵ ، مسیر محاسباتی مربوط به μ و $\log \sigma^2$ همچنان قابل‌مشتق‌گیری باقی می‌ماند.

برای غنی‌تر شدن انکودر نسبت به نسخه ساده، لایه‌های BatchNormalization و Dropout با نرخ ۲۵٪ نیز به معماری افزوده شدند (`use_batch_norm=True`) که به تثبیت آموزش و جلوگیری از بیش‌برازش کمک می‌کنند. دیکودر بدون تغییر و دقیقاً همان دیکودر اتوانکودر ساده مورد استفاده مجدد قرار گرفت.

◀ تابع هزینه و آموزش VAE

تابع هزینه VAE از دو جمله تشکیل شده است: جمله بازسازی (مشابه اتوانکودر ساده) و جمله KL Divergence که فاصله توزیع نهفته تولیدشده توسط انکودر را از توزیع نرمال استاندارد $\mathcal{N}(\mathbf{0}, I)$ اندازه می‌گیرد:

$$\mathcal{L}_{KL} = -\frac{1}{2} \sum_{j=1}^Z \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2 \right)$$

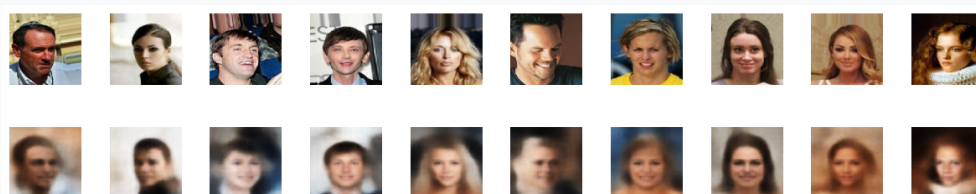
جمله بازسازی نیز با یک ضریب $\text{LOSS_FACTOR} = 10000$ وزن دهی شد تا اهمیت آن در برابر جمله KL که مقادیر کوچکتری دارد، حفظ شود:

$$\mathcal{L}_{total} = \text{LOSS_FACTOR} \times \mathcal{L}_r + \mathcal{L}_{KL}$$

نقش جمله KL Divergence این است که فضای نهفته را وادار می‌کند به جای پراکنده و ناپیوسته بودن (مانند اتوانکودر ساده)، حول مبدأ و به شکل یک توزیع نرمال استاندارد و پیوسته سازمان‌دهی شود. همین ویژگی است که در ادامه امکان تولید نمونه‌های جدید و معنادار را با نمونه‌برداری تصادفی از $\mathcal{N}(0, I)$ فراهم می‌کند. مدل با همان تنظیمات بهینه‌ساز Adam، نرخ یادگیری 0.0005 و به مدت 10 دوره آموزش داده شد.

◀ نتایج بازسازی تصاویر با VAE

شکل زیر نتیجه بازسازی تصاویر توسط VAE را نشان می‌دهد.

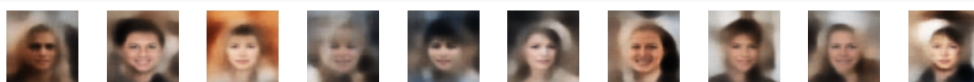


شکل ۱۹: ردیف بالا: تصاویر اصلی، ردیف پایین: تصاویر بازسازی‌شده توسط VAE

در مقایسه با اتوانکودر ساده، تصاویر بازسازی‌شده توسط VAE معمولاً اندکی محوتر به نظر می‌رسند. این موضوع طبیعی است، چرا که مدل مجبور است بخشی از ظرفیت خود را صرف برآورده کردن قید KL Divergence کند، به جای آنکه تمام تمرکز خود را صرفاً روی کمینه‌کردن خطای بازسازی بگذارد. به بیان دیگر، بین کیفیت بازسازی و منظم بودن (regularity) فضای نهفته یک trade-off وجود دارد که با پارامتر LOSS_FACTOR کنترل می‌شود.

◀ تولید چهره‌های جدید از فضای نهفته

مزیت اصلی VAE نسبت به اتوانکودر ساده، امکان تولید نمونه‌های کاملاً جدید است. با نمونه‌برداری از یک توزیع نرمال استاندارد $z \sim \mathcal{N}(0, I)$ در فضای ۲۰۰ بعدی و عبور آن از دیکودر آموزش‌دیده، تصاویری از چهره‌های جدید (که در مجموعه داده اصلی وجود نداشتند) تولید شدند.



شکل ۲۰: چهره‌های جدید تولیدشده با نمونه‌برداری تصادفی از فضای نهفته VAE

این نتیجه به خوبی نشان می‌دهد که به لطف جمله KL Divergence، فضای نهفته VAE به یک فضای پیوسته و متراکم حول مبدأ تبدیل شده که هر نقطه دلخواه در آن، به یک تصویر معنادار و شبیه به چهره انسان نگاشت می‌شود. این در حالی است که تلاش برای انجام همین کار با اتوانکودر ساده (یعنی نمونه‌برداری تصادفی از فضای نهفته آن) معمولاً به تصاویری بی‌معنا و نامفهوم منجر می‌شود؛ زیرا هیچ قیدی روی توزیع بردارهای نهفته در اتوانکودر ساده اعمال نشده و فضای نهفته آن می‌تواند خالی، ناپیوسته و پر از حفره باشد.

مقایسه اتوانکودر ساده و VAE و اثر تغییر LOSS_FACTOR

با مقایسه دو مدل می‌توان جمع‌بندی زیر را ارائه داد:

- از نظر کیفیت خام بازسازی (Reconstruction Quality)، اتوانکودر ساده معمولاً عملکرد اندکی بهتر و تصاویر شارپ‌تری نسبت به VAE ارائه می‌دهد، زیرا هیچ قیدی روی فضای نهفته آن اعمال نشده و تمام ظرفیت مدل صرف کمینه‌سازی خطای بازسازی می‌شود.
- از نظر کیفیت و کاربردپذیری فضای نهفته، VAE به‌طور قطع برتری دارد، چون فضای نهفته آن پیوسته، متراکم و قابل نمونه‌برداری است و امکان تولید داده جدید (Generative capability) را فراهم می‌کند؛ قابلیت‌هایی که اتوانکودر ساده اساساً فاقد آن است.
- پارامتر LOSS_FACTOR نقش تنظیم این trade-off را دارد: مقادیر بزرگ‌تر آن، وزن جمله بازسازی را افزایش داده و رفتار مدل را به سمت یک اتوانکودر معمولی

(بازسازی شارپتر، اما فضای نهفته نامنظمتر) سوق می‌دهد؛ در مقابل مقادیر کوچکتر آن، جمله KL را پررنگتر کرده و فضای نهفته را منظمتر می‌کند، اما به قیمت محوتر شدن بیشتر تصاویر بازسازی‌شده.

با توجه به هدف اصلی این تمرین که تولید چهره‌های جدید بود، **مدل VAE سودمندتر** از اتوانکودر ساده ارزیابی می‌شود؛ چرا که با وجود افت جزئی در وضوح بازسازی، قابلیت منحصربه‌فرد تولید نمونه‌های جدید و معنادار را در اختیار قرار می‌دهد که کاربرد اصلی و اهمیت مدل‌های مولد را نشان می‌دهد.

جمع‌بندی

در این تمرین ابتدا یک اتوانکودر ساده و سپس نسخه واریشنال آن روی مجموعه داده CelebA پیاده‌سازی و آموزش داده شد. نتایج نشان داد که هرچند اتوانکودر ساده در بازسازی تصاویر ورودی عملکرد نسبتاً خوبی دارد، اما فاقد ساختار مناسب برای تولید داده جدید است. در مقابل، VAE با افزودن قید KL Divergence و مکانیزم Reparameterization، فضای نهفته‌ای منظم و پیوسته می‌سازد که امکان تولید چهره‌های جدید و باورپذیر را با نمونه‌برداری تصادفی از توزیع نرمال استاندارد فراهم می‌کند. این آزمایش به‌خوبی تفاوت بنیادین میان یک مدل فشرده‌ساز صرف (اتوانکودر ساده) و یک مدل مولد واقعی (VAE) را نشان می‌دهد.

۱۰ تمرین سوم - سوال دوم: ترکیب Variational Autoencoders و Normalizing Flows

هدف مساله

در این سوال قصد داریم با استفاده از مفهوم جریان‌های نرمال‌ساز (Normalizing Flows)، انعطاف‌پذیری و قدرت مدل‌سازی فضای پنهان (Latent Space) را در مدل‌های VAE افزایش دهیم. در VAE استاندارد، توزیع پیشین (Prior) معمولاً یک توزیع گاوسی ساده فرض می‌شود که برای بازنمایی داده‌های پیچیده محدودیت ایجاد می‌کند. با اعمال دنباله‌ای از نگاشت‌های معکوس‌پذیر، این توزیع ساده به توزیع‌های پیچیده‌تر و چندوجهی تبدیل می‌شود. در این تمرین، عملکرد این ایده روی مجموعه داده FashionMNIST مورد بررسی و ارزیابی قرار گرفته است.

بصری‌سازی داده‌های ورودی (FashionMNIST)

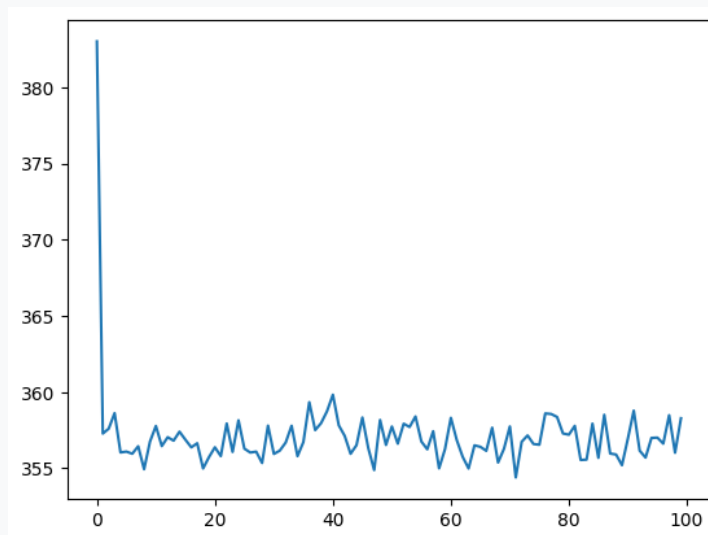
در ابتدای کار، یک بچ (Batch) ثابت از داده‌های تست برای ارزیابی کیفی مدل‌ها انتخاب شد. مجموعه داده FashionMNIST شامل تصاویر 28×28 پیکسل سیاه‌وسفید از پوشاک (نظیر کفش، کیف، تیشرت و...) است. شکل زیر نمونه‌ای از این داده‌های ورودی را نشان می‌دهد که قرار است مدل‌های ما یاد بگیرند آن‌ها را بازسازی کنند.



شکل ۲۱: نمونه‌ای از داده‌های ورودی مجموعه FashionMNIST

تحلیل روند آموزش و خطای بازسازی (Loss)

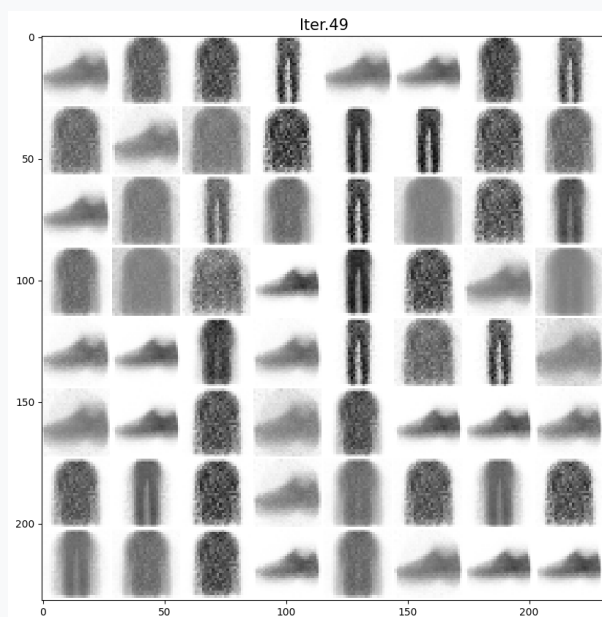
در فرآیند آموزش مدل‌ها، تابع هزینه شامل دو بخش اصلی است: خطای بازسازی (images/Reconstruction Loss) برای سنجش کیفیت تصاویر تولید شده، و یک عبارت تنظیم‌کننده (مانند Divergence KL یا Free Variational Energy) برای ساختاردهی به فضای پنهان. همان‌طور که در نمودار زیر مشاهده می‌شود، خطای آموزش در اپوک‌های اولیه با شیب تندی کاهش یافته و سپس به یک پایداری نسبی می‌رسد. نوسانات جزئی در نمودار به دلیل استفاده از زیرمجموعه‌های تصادفی (Subsampling) در هر اپوک است.



شکل ۲۲: روند کاهش خطای بازسازی در طول تکرارهای آموزش

کیفیت بازسازی تصاویر (Reconstructions)

در طول فرآیند آموزش، مدل‌ها سعی کردند تصاویر بیچ ثابت (Fixed Batch) را بازسازی کنند. در تکرارهای اولیه، خروجی مدل‌ها صرفاً هاله‌ای تار و نویزدار از تصاویر بود. اما با پیشرفت آموزش، مدل‌ها توانستند ساختار کلی اشیاء (مانند فرم کلی کفش یا کیف) را یاد بگیرند. در مقایسه بین مدل‌ها، مدل‌هایی که به جریان نرمال‌ساز مجهز بودند (Planar Flow)، توانستند جزئیات و لبه‌های تصاویر را با وضوح بهتری نسبت به Vanilla VAE بازسازی کنند.



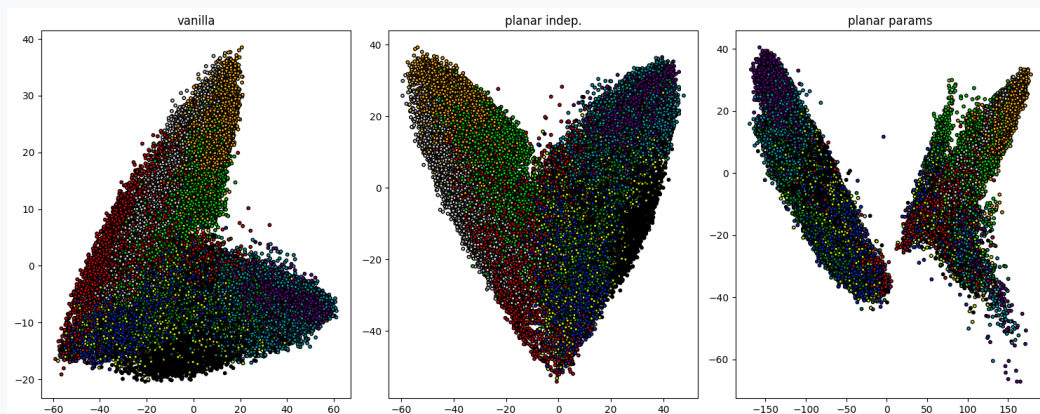
شکل ۲۳: خروجی بازسازی شده تصاویر توسط مدل پس از اتمام آموزش

تحلیل فضای پنهان (Latent Space) و مقایسه سه مدل

مهم‌ترین بخش این تمرین، بررسی تاثیر افزودن Flow به فضای پنهان است. برای این منظور، سه ساختار مختلف پیاده‌سازی شدند و خروجی آن‌ها توسط الگوریتم کاهش ابعاد PCA (به دو بُعد) تصویرسازی شد:

- **حالت اول - Vanilla VAE:** در این حالت (نمودار سمت چپ)، به دلیل استفاده از یک توزیع نرمال ساده، کلاس‌های مختلف به شدت درهم‌تنیده هستند و مرز مشخصی بین کلاسترها وجود ندارد. مدل در جداسازی ویژگی‌های پوشاک مختلف ضعیف عمل کرده است.
- **حالت دوم - Planar Independent:** با اضافه شدن جریان‌های متوالی به توزیع پنهان (نمودار وسط)، توزیع از حالت گاوسی خارج شده و منعطف‌تر شده است. در اینجا کلاسترها (گروه‌های رنگی مشابه) فشردگی بهتری پیدا کرده‌اند و تداخل کلاس‌های نامرتب با حد قابل توجهی کاهش یافته است.
- **حالت سوم (نمره امتیازی) - Planar Params / Amortized Inference:** در این حالت پیشرفته (نمودار سمت راست)، پارامترهای جریان نرمال‌ساز ایستا

نیستند و شبکه‌ی انکودر متناسب با هر تصویر، پارامترهای اختصاصی Flow مربوط به آن را پیش‌بینی می‌کند. همان‌طور که مشخص است، این حالت بهترین تفکیک‌پذیری را ایجاد کرده است. کلاس‌های هم‌خانواده با تراکم بسیار بالا در کلاسترهای کاملاً مجزا و جزای قرار گرفته‌اند. همچنین ارزیابی NLL (Negative Log-Likelihood) نیز نشان می‌دهد که این مدل کمترین میزان خطا را در مدل‌سازی توزیع داده‌ها داشته است.



شکل ۲۴: مقایسه فضای پنهان استخراج شده توسط مدل‌های مختلف با استفاده از الگوریتم PCA

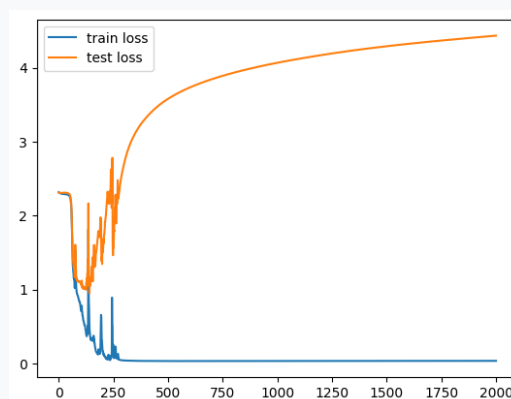
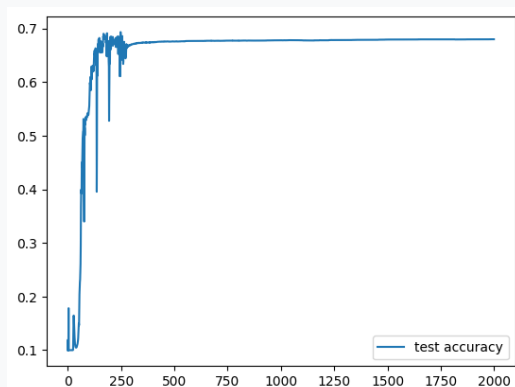
۱۱ تمرین سوم - سوال سوم: یادگیری خودنظارتی (Self-Supervised Learning) روی MNIST Fashion

هدف و معرفی مسئله

هدف از این تمرین، پیاده‌سازی یک مدل یادگیری خودنظارتی (Self-Supervised Learning) بر روی مجموعه داده Fashion MNIST است. در بسیاری از مسائل واقعی، دسترسی به داده‌های برچسب‌دار بسیار پرهزینه است. در این سناریو فرض شده است که از ۶۰۰۰۰ داده‌ی آموزشی، تنها ۲۰۰ داده دارای برچسب هستند و ۵۹۸۰۰ داده‌ی دیگر بدون برچسب رها شده‌اند. هدف این است که با استفاده از تکنیک‌های یادگیری تقابلی (Contrastive Learning)، ویژگی‌های معنادار تصاویر استخراج شده و سپس مدل صرفاً با آن ۲۰۰ داده برچسب‌دار تنظیم دقیق (Fine-tune) شود.

مدل پایه (Supervised Baseline) و چالش بیش‌برازش

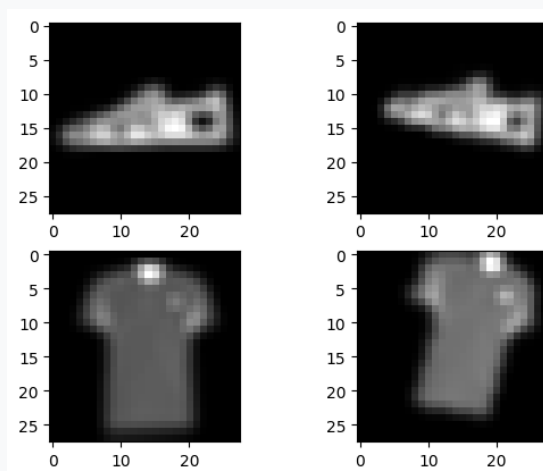
در مرحله‌ی اول، یک مدل پایه شامل یک Backbone کانولوشنی و یک Head تمام‌متصل تعریف شد و صرفاً بر روی همان ۲۰۰ داده‌ی برچسب‌دار آموزش دید. همان‌طور که در نمودارهای زیر مشخص است، به دلیل کمبود شدید داده‌های آموزشی، مدل به سرعت دچار بیش‌برازش (Overfitting) می‌شود؛ خطای آموزش (Train Loss) به صفر نزدیک شده، اما خطای ارزیابی (Test Loss) به شدت افزایش یافته و دقت مدل در یک مقدار پایین متوقف می‌شود.



شکل ۲۵: سمت راست: دقت ارزیابی مدل پایه / سمت چپ: نمودار خطای آموزش و ارزیابی (وقوع بیش‌برازش شدید)

➤ افزودنی داده‌ها (Data Augmentation) برای یادگیری تقابلی

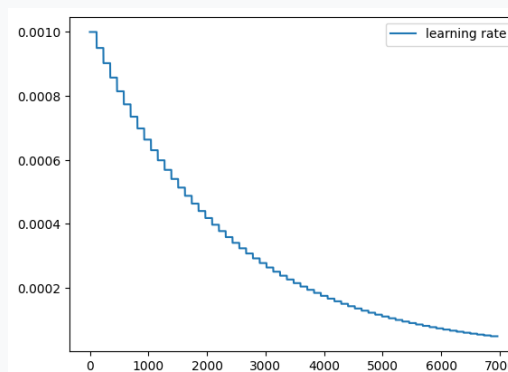
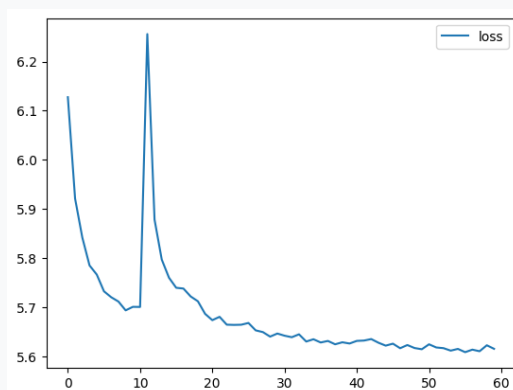
برای استفاده از ۵۹۸۰۰ داده‌ی بدون برچسب، باید جفت‌های مثبت (Positive Pairs) از تصاویر تولید می‌کردیم. بدین منظور از رویکرد افزودنی داده‌ها با اعمال تغییرات تصادفی هندسی و بصری نظیر Random Affine، Random Perspective، Gaussian و Blur استفاده گردید. در شکل زیر، نمونه‌هایی از جفت‌های مثبت تولید شده (مانند یک کفش یا تیشرت که با زاویه و تاری متفاوت دیده می‌شوند) قابل مشاهده است:



شکل ۲۶: نمونه‌ای از جفت‌های مثبت تولید شده جهت آموزش بدون نظارت

➤ آموزش خودنظارتی با تابع هزینه Contrastive

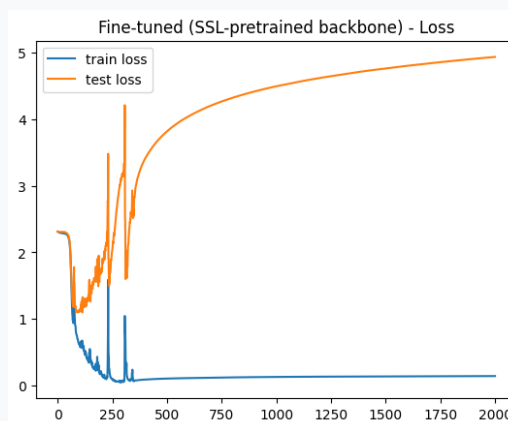
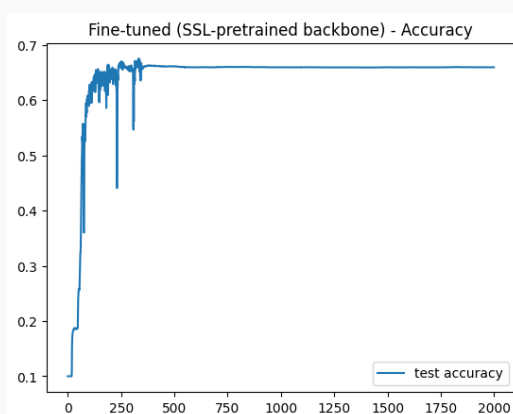
شبکه Backbone بر روی داده‌های بدون برچسب آموزش داده شد. در این فاز از تابع هزینه Contrastive Loss استفاده گردید تا مدل یاد بگیرد فاصله بردارهای ویژگی جفت‌های مثبت را کاهش و فاصله آن‌ها با سایر داده‌های درون Batch (جفت‌های منفی) را افزایش دهد. برای بهبود همگرایی، از زمان‌بندی نمایی نرخ یادگیری (Exponential LR Scheduler) استفاده شد. نمودارها نشان می‌دهند که تابع هزینه در طول دوره‌های آموزشی روند کاهشی مطلوبی داشته و مدل توانسته ویژگی‌های ساختاری داده‌ها را به خوبی درک کند.



شکل ۲۷: سمت راست: روند کاهششی تابع هزینه Contrastive / سمت چپ: کاهش نمایی نرخ یادگیری

تنظیم دقیق (Fine-Tuning) و نتیجه‌گیری نهایی

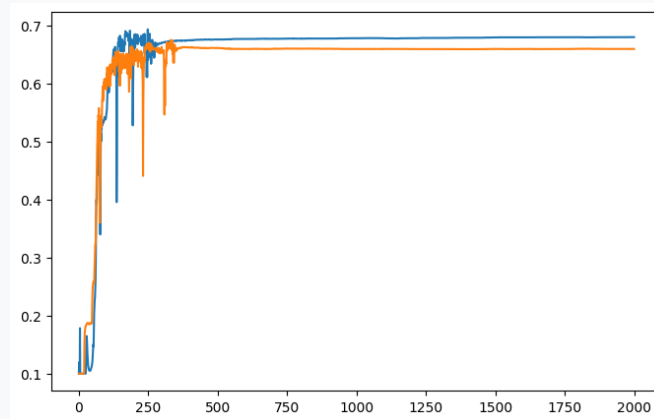
پس از پایان پیش‌آموزش بدون نظارت، وزن‌های آموزش‌دیده در بخش Backbone به عنوان نقطه شروع در نظر گرفته شدند و مدل نهایی مجدداً بر روی همان ۲۰۰ داده‌ی برچسب‌دار تنظیم دقیق (Fine-tune) گردید.



شکل ۲۸: نمودارهای خطا و دقت پس از Fine-Tuning با استفاده از وزن‌های از پیش‌آموزش‌دیده

مقایسه و جمع‌بندی نهایی: نمودار زیر، مقایسه دقت مدل پایه (بدون پیش‌آموزش) و مدل بهره‌مند از یادگیری خودنظارتی را نشان می‌دهد. نتایج به وضوح ثابت می‌کنند مدلی که ابتدا به صورت خودنظارتی ویژگی‌های ساختاری داده‌ها را یاد گرفته باشد،

در مواجهه با داده‌های به شدت محدود (تنها ۲۰۰ نمونه)، عملکرد پایدارتر و توانایی تعمیم (Generalization) بسیار بهتری از خود نشان می‌دهد و از شدت بیش‌برازش کاسته می‌شود.



شکل ۲۹: مقایسه دقت مدل پایه (آبی) با مدل خودنظارتی (نارنجی)